Predictive AI Asset Maintenance System

DBMS Laboratory Project

Student 1: Vinit Karnkar

Student 2: Project Partner

Domain: IT Infrastructure

AI/DS Integration: Failure Prediction / Predictive Maintenance

Abstract

The Predictive AI Asset Maintenance System is a local web application that combines relational database management, SQL analytics and machine learning to support proactive IT infrastructure maintenance. Assets, locations, technicians, telemetry, failures, maintenance activities and prediction history are stored in a normalized MySQL database. A Flask application provides an enterprise-style monitoring dashboard. A machine-learning pipeline compares Logistic Regression and Random Forest and stores failure-risk predictions back into the database.

Problem Statement

Reactive maintenance can lead to downtime, repair expense and service disruption. Organizations need a consolidated view of asset health and a mechanism to prioritize maintenance using historical evidence.

Proposed System

The proposed system centralizes infrastructure data, applies advanced SQL analytics, and estimates failure probability for assets using operational and historical features.

Objectives

• Manage IT infrastructure assets and relationships.

• Store historical telemetry, maintenance and failure events.

• Demonstrate normalization and advanced SQL.

• Automate operational workflows with procedures and triggers.

• Train and evaluate classification models.

• Present health/risk insights through a professional dashboard.

Scope

The academic implementation covers servers, network devices, storage, workstations, printers, maintenance, failures and synthetic operational metrics. It is designed for local demonstration.

Corporate Relevance

The same architecture can support enterprise asset inventories and maintenance teams. Production deployment would require real telemetry, authenticated integrations, stronger model validation and operational security.

System Architecture

User → Web Interface → Flask Backend → MySQL. Historical data flows to analytics and ML feature engineering. Predictions are persisted to MySQL and rendered on the dashboard.

See `diagrams/system_architecture.png`.

Database Design

Nine core tables are implemented: users, locations, technicians, assets, maintenance, asset_metrics, failures, predictions and maintenance_schedule.

Normalization

The database is designed through 1NF, 2NF and 3NF principles. See `normalization.md`.

SQL/DBMS Implementation

The project includes primary/foreign keys, constraints, indexes, views, stored procedures, triggers, CTEs, window functions, subqueries, conditional aggregation and date analytics.

Machine Learning Methodology

The target is a binary future-failure indicator for the academic dataset. Features include asset age, CPU/memory/disk utilization, temperature, network load, error count, uptime, prior failures, maintenance gap/count/cost and criticality.

Two models are trained:

• Logistic Regression

• Random Forest

Evaluation uses accuracy, precision, recall, F1, ROC-AUC and a confusion matrix. Model selection prioritizes F1 and ROC-AUC because failure detection is imbalanced.

Prediction Workflow

1. Retrieve latest metrics and historical aggregates.

2. Engineer the model feature vector.

3. Load the selected Joblib model.

4. Generate failure probability.

5. Convert probability to LOW/MEDIUM/HIGH risk.

6. Calculate a separate health score.

7. Generate a maintenance recommendation.

8. Store the prediction in MySQL.

Results

The dashboard presents asset inventory, health/risk distribution, failure trends, maintenance cost and operational alerts. Exact model metrics are generated by `ml/train_model.py` from the installed seed data.

Advantages

• Strong DBMS implementation.

• Traceable prediction history.

• Automated trigger-based maintenance workflow.

• Clear operational risk visualization.

• Fully local and low-cost setup.

Limitations

The dataset is synthetic and intended for an academic demonstration. The failure target is not a substitute for production incident labeling. The health score is a project-defined indicator rather than a certified reliability metric.

Future Scope

• Real monitoring/telemetry ingestion.

• More rigorous time-based validation.

• SHAP explanations.

• Automated email/SMS/ITSM alerts.

• Expanded role-specific authorization and audit logging.

• Model drift monitoring.

• Containerized/cloud deployment.

Conclusion

The system demonstrates how DBMS concepts can form the foundation of an AI-enhanced enterprise application. MySQL provides structured history and analytical capabilities while Flask and machine learning convert that information into actionable maintenance intelligence.

Security hardening and ML integrity

The revised implementation uses password hashing, CSRF protection, role-based authorization, and environment-based configuration. The ML training dataset is constructed from historical telemetry snapshots; feature values only use information available at or before the snapshot timestamp, while the label represents a failure in the following 30 days. This prevents target leakage.